

WE HAVE A DEMO

A WIRELESS PROTOCOL HACKING FRAMEWORK

Romain Cayre + Damien Cauquil



/WHO

Romain Cayre

- Author of *Mirage*, *WazaBee* & [WHAD](#)
- Teaching cybersecurity at **INSA Toulouse**

Damien Cauquil

- Author of *Btlejack*, *BumbleBee* & [WHAD](#)
- Security engineer at **Quarkslab**

LET'S TALK A BIT ABOUT *WHAD*...



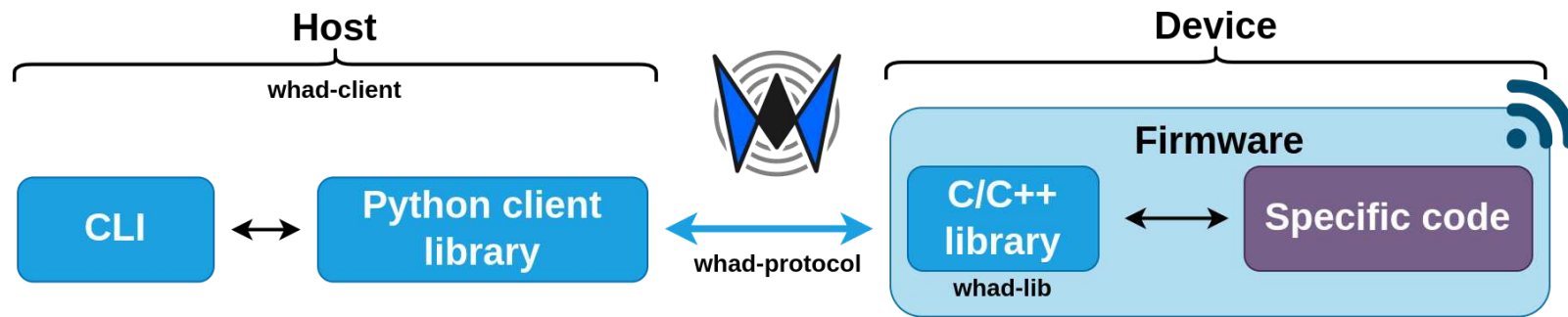
- Wireless HAcking for Dummies 🤔
 - Unified *Host/Hardware* protocol
 - Python API + *off-the-shelf* tools
 - Development started in 2021...
 - Project released in 2024

<https://whad.io>

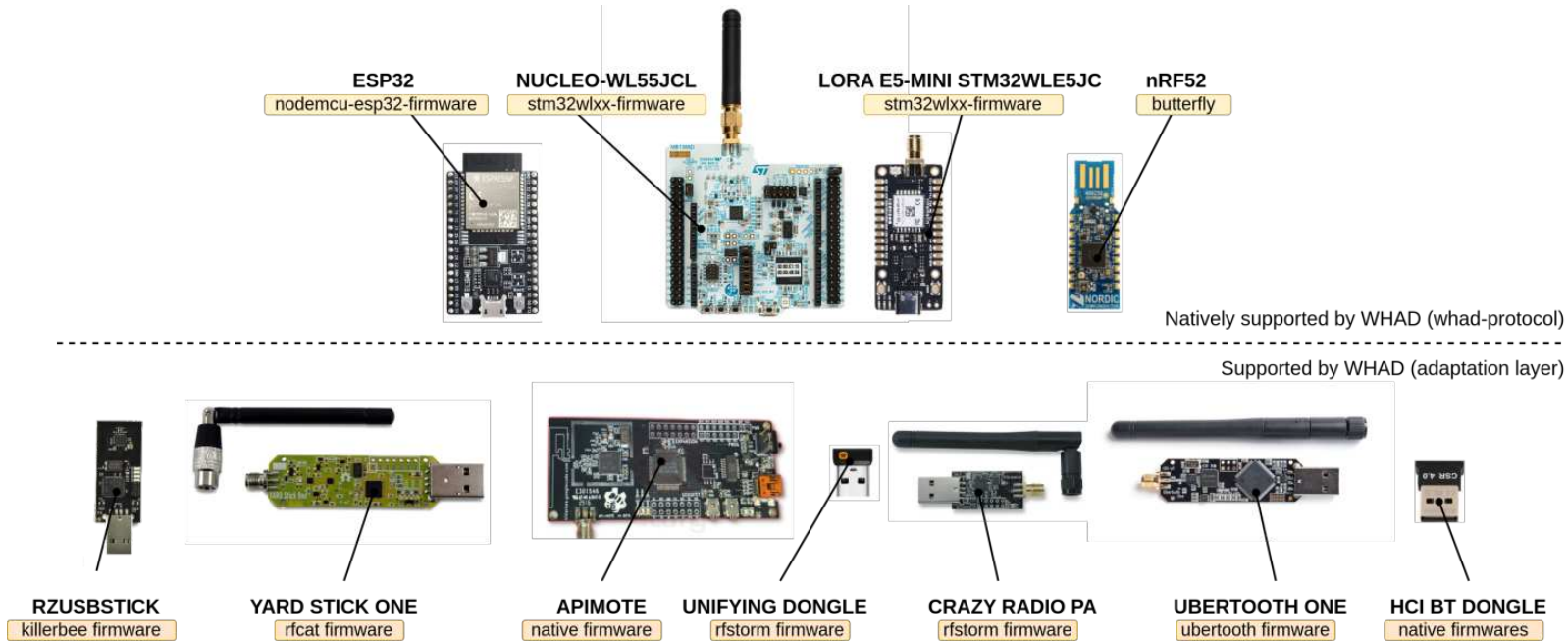
WHY WHAD ?

- **Portability:** design once, use with any compatible hardware
- **Affordability:** relies on inexpensive *off-the-shelf* hardware
- **Flexibility:** Python and Scapy enables **rapid prototyping**

OVERVIEW



COMPATIBLE HARDWARE



SUPPORTED PROTOCOLS

- SB / ESB / Logitech Unifying
- ANT / ANT+ / ANTFS
- IEEE 802.15.4 / ZigBee / RF4CE / Wireless HART
- LoRa / LoRaWAN
- Bluetooth Low Energy 4.x (*support for 5.x soon*)

WHAD ❤️ SCAPY

- Every PDU, packet or frame is a Scapy packet
- We added support for various protocols in **Scapy** (not upstreamed yet 🙄)
- Some of our tools allow packet manipulation with user-supplied Python code



WHAD SCAPY - EXAMPLES

Example of IEEE802.15.4-TAP (DLT=287)

PhilipsLight_01:04:...	0x0001	IEEE 802.15.4	65 Association Request, FFD
PhilipsLight_01:05:...	PhilipsLight_01:04:...	IEEE 802.15.4	71 Association Response, PAN: 0x3180 Addr: 0x0004
0x0001		ZigBee	72 Beacon, Src: 0x0001, EPID: 09:40:84:3f:53:27:00:
0x0001	0x0004	ZigBee	89 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0004	ZigBee	89 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0004	ZigBee	89 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0004	ZigBee	89 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0001	ZigBee	89 Data, Dst: 0x0001, Src: 0x0004
0x0001	0x0004	ZigBee	89 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0004	ZigBee	91 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0004	ZigBee	92 Data, Dst: 0x0004, Src: 0x0001
0x0001	0x0004	ZigBee	92 Data, Dst: 0x0004, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001
0x0001	Broadcast	ZigBee	92 Data, Dst: Broadcast, Src: 0x0001

Frame 3: Packet, 65 bytes on wire (520 bits), 65 bytes captured (520 bits) on interface /tmp/1rvb59ng.pcap, id 0

IEEE 802.15.4 TAP

Header

RSS: -45,00 dBm

Link Quality Indicator: 192

Channel assignment: Page: Default (0), Number: 25

Channel center frequency: 2475000070144,000 kHz

FCS type: ITU-T CRC16 (1)

[Data Length: 21]

IEEE 802.15.4 Command, Src: PhilipsLight_01:04:b9:d1:33, Dst: 0x0001

Frame Control Field: 0xc823, Frame Type: Command, Acknowledge Request, Destination Addressing Mode: Short/16-bit, Frame Version:

Sequence Number: 108

Destination PAN: 0x3180

Destination: 0x0001

Source PAN: 0xffff

Extended Source: PhilipsLight_01:04:b9:d1:33 (00:17:88:01:04:b9:d1:33)

Command Identifier: Association Request (0x01)

Association Request

FCS: 0x2f60 (Correct)

WHAD SCAPY - EXAMPLES

Communication with RF hardware

```

bind_layers(GoodFET_Command_Hdr, GoodFET_Peek_Command, verb = 0x02)
bind_layers(GoodFET_Command_Hdr, GoodFET_Monitor_Connected_Command, verb = 0xB1)
bind_layers(GoodFET_Command_Hdr, GoodFET_Setup_CSPI_Command, verb = 0x10)
bind_layers(ANTStick_Message, ANTStick_Command_Set_Channel_Period, id=0x43)
bind_layers(ANTStick_Message, ANTStick_Command_Unassign_Channel, id=0x41)
bind_layers(ANTStick_Message, ANTStick_Command_Assign_Channel, id=0x42)
bind_layers(ANTStick_Message, ANTStick_Command_Search_Timeout, id=0x44)
bind_layers(ANTStick_Message, ANTStick_Command_Set_Channel_RF_Frequency, id=0x45)
bind_layers(ANTStick_Message, ANTStick_Command_Set_Network_Key, id=0x46)
bind_layers(ANTStick_Message, ANTStick_Command_Reset, id=0x4A)
bind_layers(ANTStick_Message, ANTStick_Command_Open_Channel, id=0x4B)
bind_layers(ANTStick_Message, ANTStick_Command_Close_Channel, id=0x4C)
bind_layers(ANTStick_Message, ANTStick_Command_Request_Message, id=0x4D)
bind_layers(ANTStick_Message, ANTStick_Command_Set_Channel_ID, id=0x51)
bind_layers(ANTStick_Message, ANTStick_Requested_Message_Capabilities, id=0x54)
bind_layers(ANTStick_Message, ANTStick_Command_Open_RX_Scan_Mode, id=0x5B)
bind_layers(ANTStick_Message, ANTStick_Requested_Message_Serial_Number, id=0x61)
bind_layers(ANTStick_Message, ANTStick_Command_Low_Priority_Search_Timeout, id=0x63)
bind_layers(ANTStick_Message, ANTStick_Command_Enable_Extended_Messages, id=0x66)
bind_layers(ANTStick_Message, ANTStick_Command_Lib_Config, id=0x6E)
bind_layers(ANTStick_Message, ANTStick_Requested_Message_Startup, id=0x6F)
bind_layers(ANTStick_Message, ANTStick_Requested_Message_Advanced_Burst, id=0x78)

```

CREATING THE RIGHT TOOLS

EXAMPLE: REAL-TIME HEART RATE MONITORING !

Implemented using 2 basic bricks !



A SYSTEMATIZATION OF WIRELESS ATTACKS

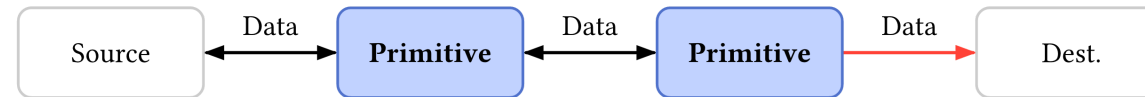
METHODOLOGY

- Definition of a **threat model**
- **Analysis & classification** of known wireless attacks
- Identification of 11 unique *primitives*

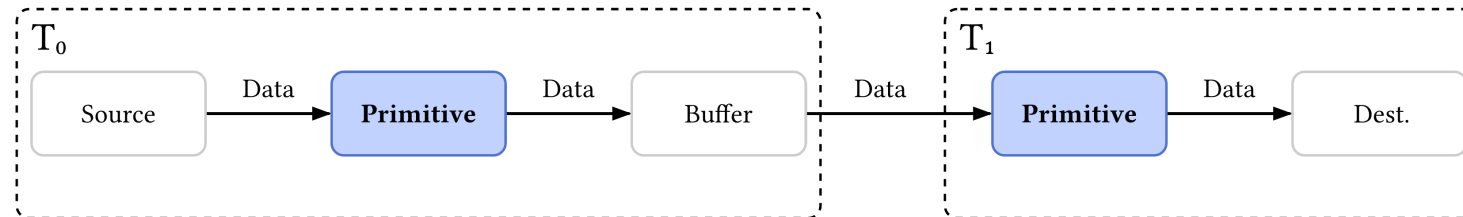
OVERVIEW OF THE PROPOSED PRIMITIVES

COMBINING PRIMITIVES

Data flow matters ...



... but also timing !



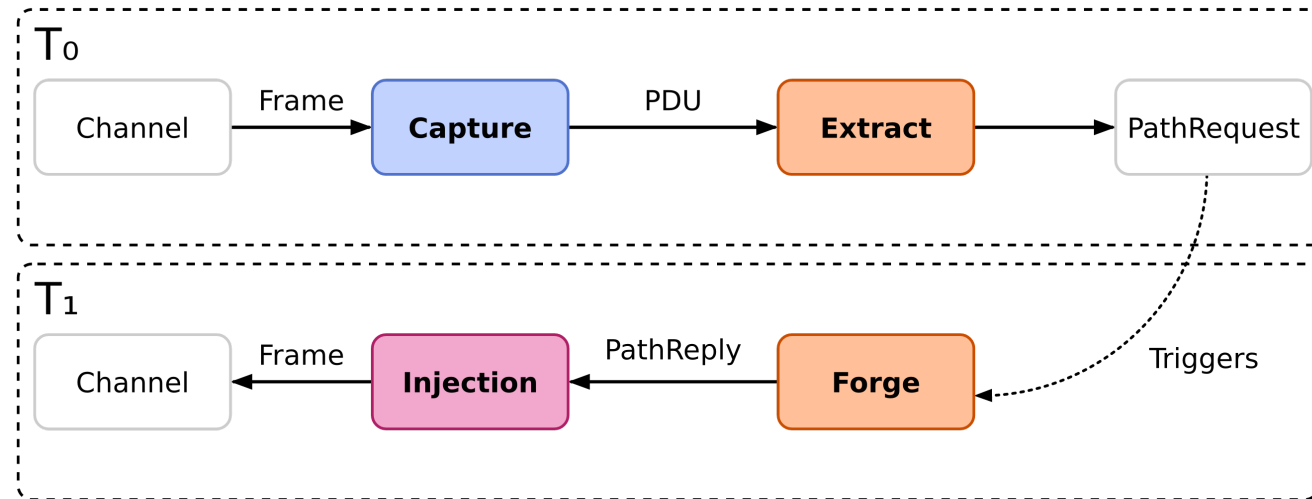
MODELING ATTACKS: SOME EXAMPLES

BLUETOOTH MESH: ARBITRARY PATH INJECTION



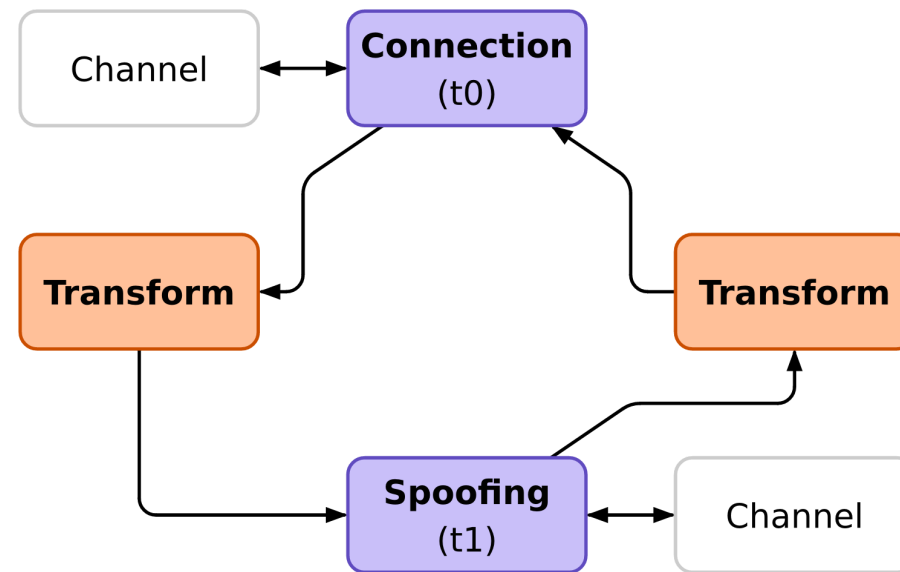
*«An attacker can create a path from any network node to another by sending a specifically crafted **PathRequest**»*

BLUETOOTH MESH: FORWARDING TABLE POISONING



«An attacker can poison a PO's forwarding table by immediately responding to a PathRequest with a spoofed PathReply, racing the legitimate node.»

BLUETOOTH LE: MAN-IN-THE-MIDDLE



«An attacker can connect to a BLE device, advertise a spoofed one, wait for a connection and tamper with data.»

LIMITS / CONSTRAINTS



Simplified representation of the physical Layer



Node roles have not been considered
to limit complexity



Abstract (& simplified) model of *networks*

FROM THEORY TO FIELD TESTING

because failure is always an option

BUILDING OUR TOOLBOX



Each **primitive** defines a **basic tool**



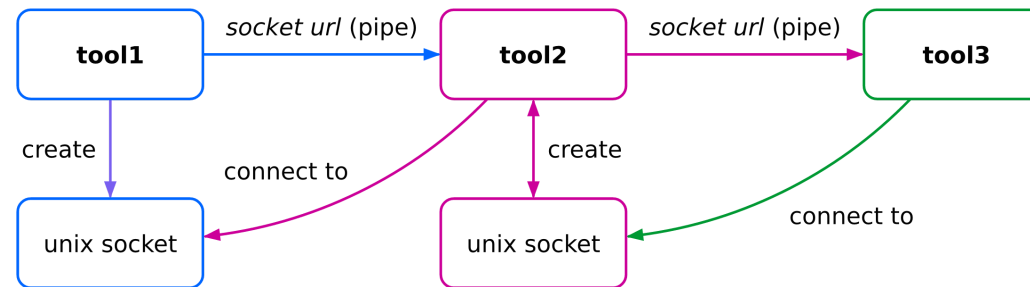
Tools can be combined to **perform complex tasks**



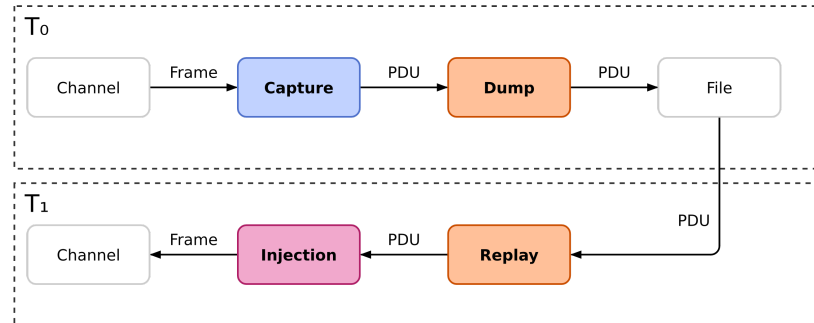
KISS: Keep It Simple Stupid !

COMBINING TOOLS

```
$ tool1 | tool2 | tool3
```

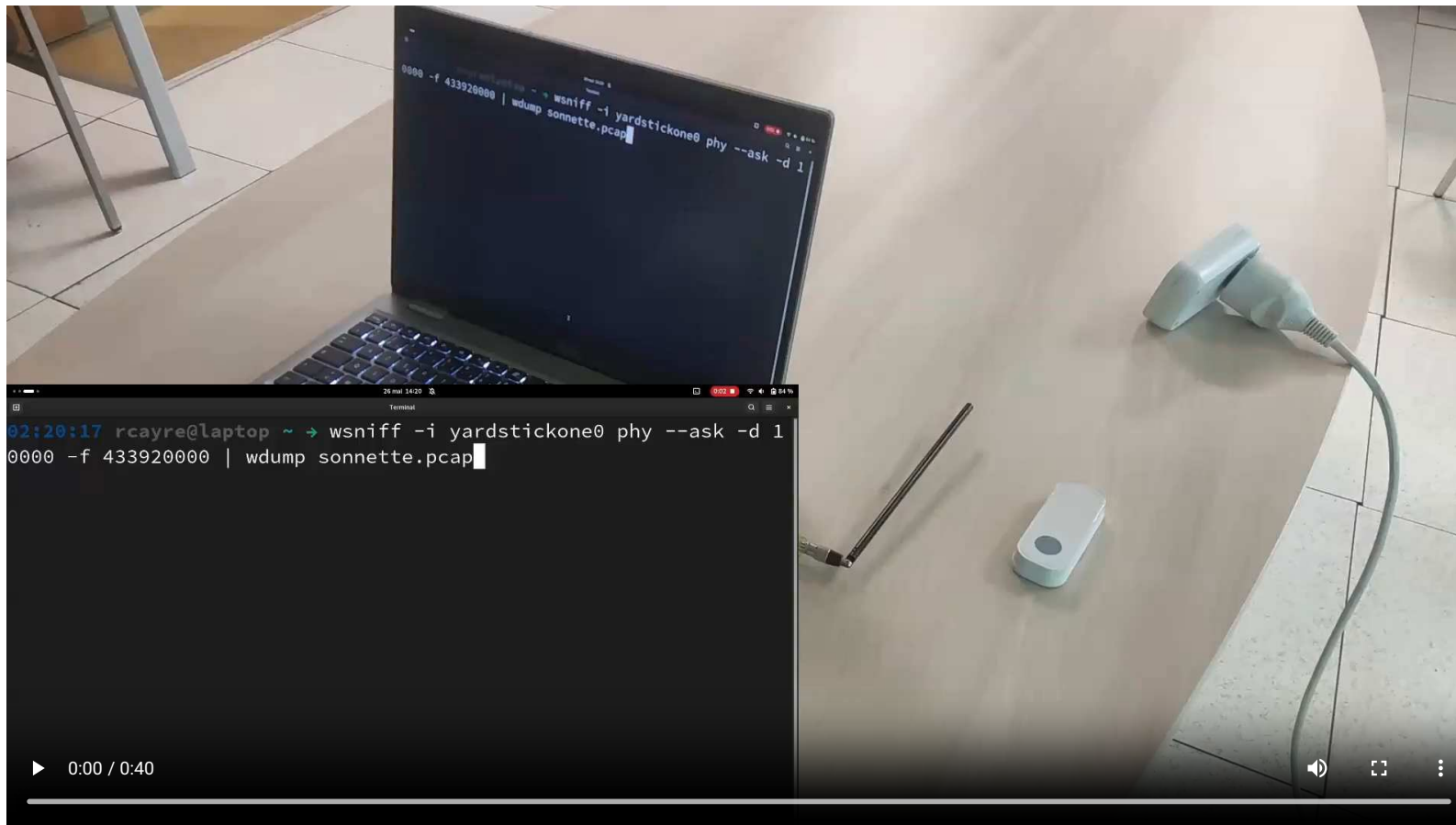


REPLAY ATTACK ON A 433MHZ DOORBELL

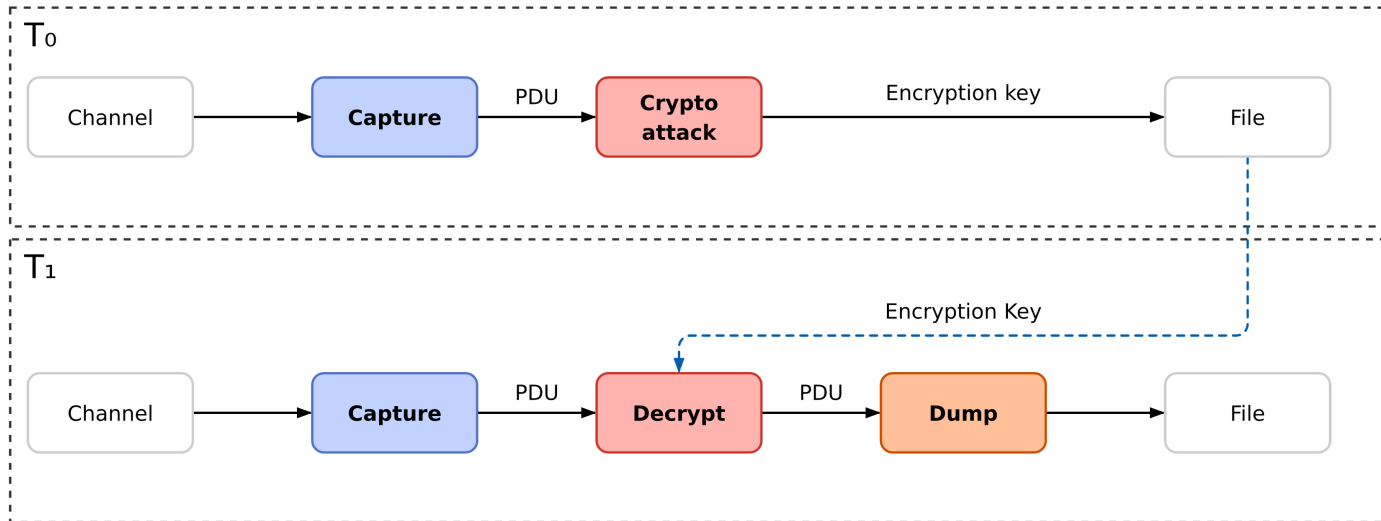


T₀: `$ wsniff -i yard0 phy -f 433092000000 --ask -d 10000 |
↳ wdump doorbell-capture.pcap`

T₁: `$ wplay doorbell-capture.pcap | winject -i yard0 phy`

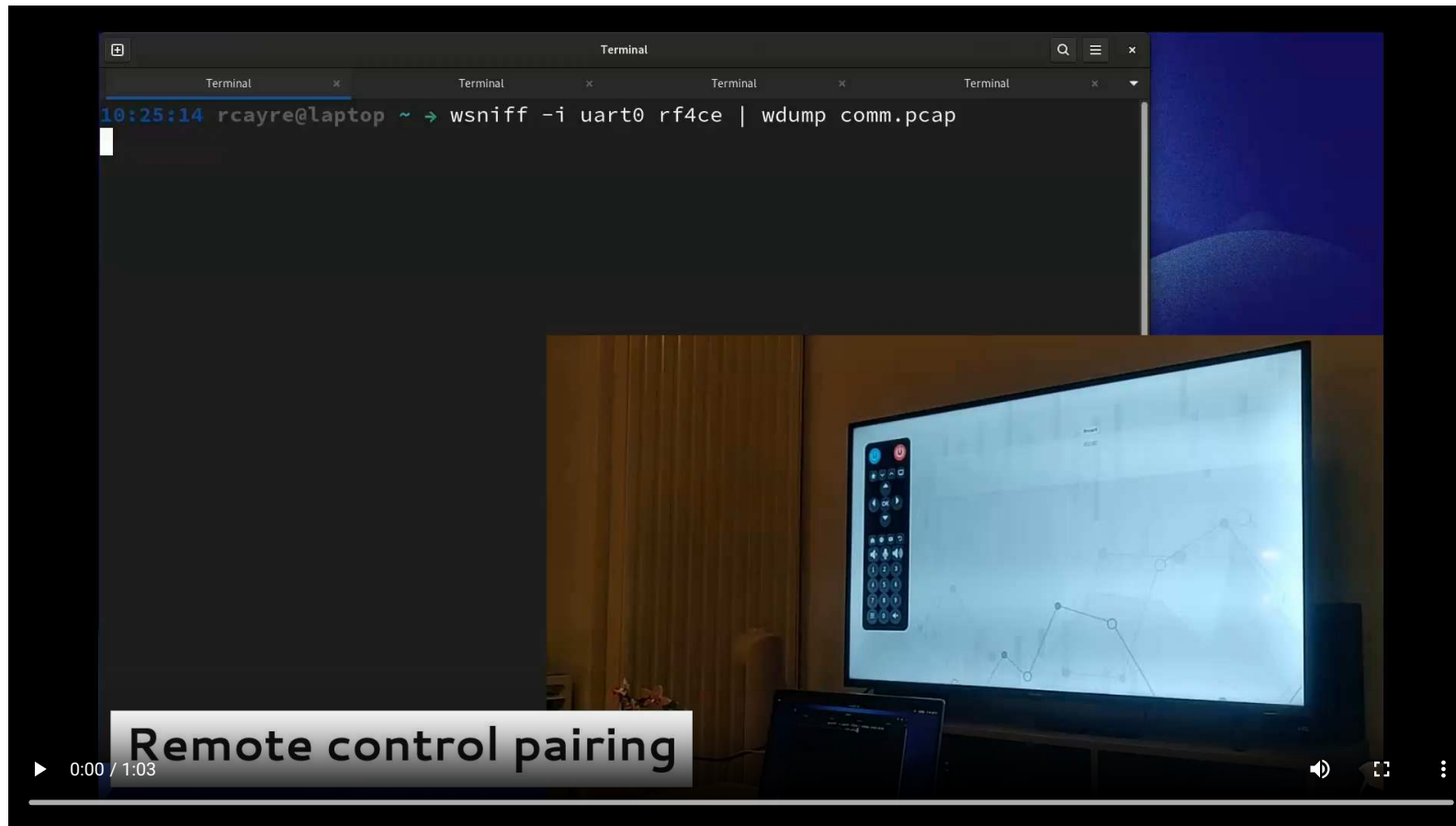


BREAKING WEAK ENCRYPTION



T₀: `$ wsniiff -i uart0 rf4ce -c 15 | wanalyze`

T₁: `$ wsniiff -i uart0 rf4ce -c 15 -d -k ENC_KEY |
↳ wdump rf4ce-cleartext.pcap`

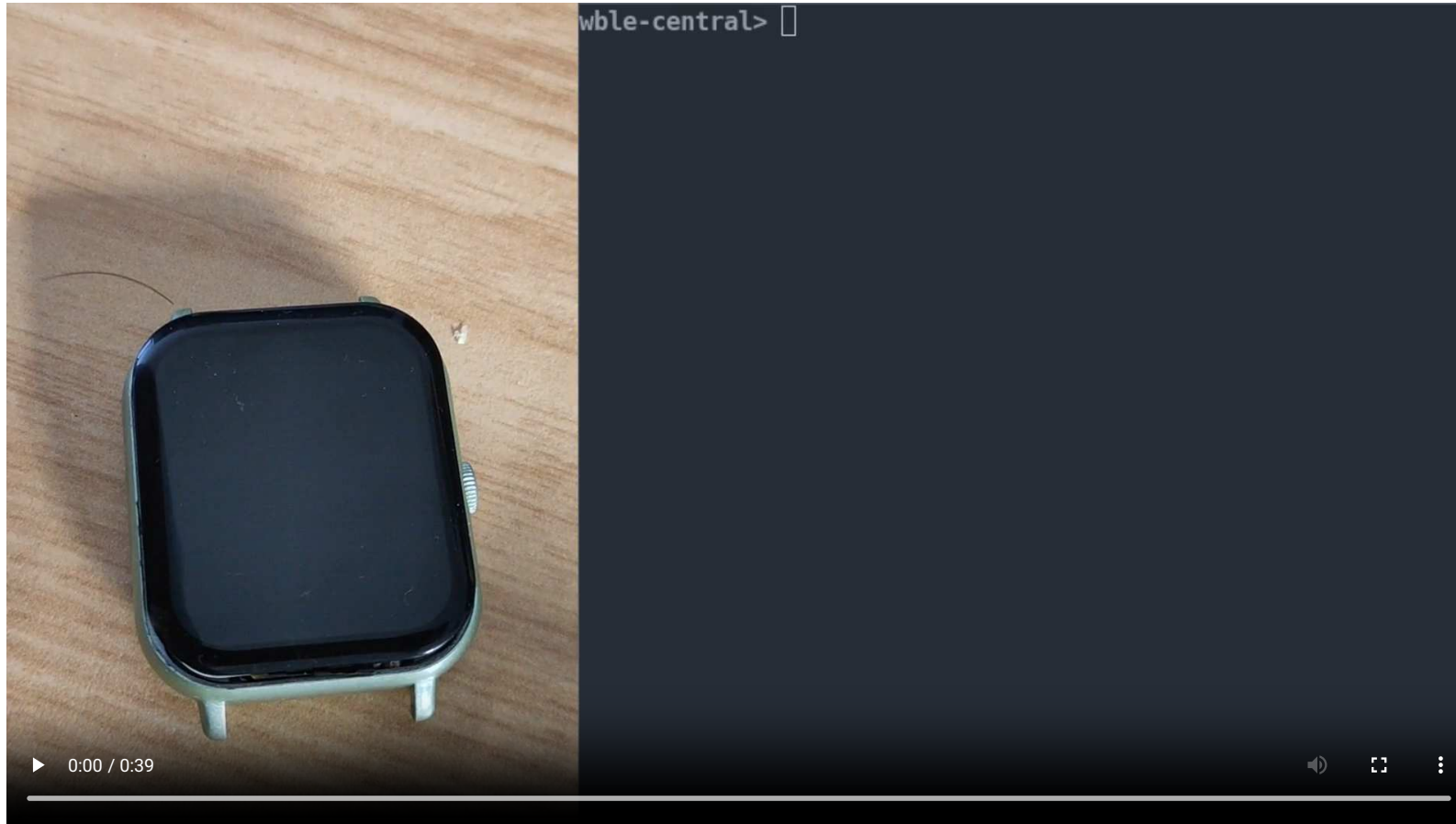


ABUSING BLE TO DUMP A FIRMWARE

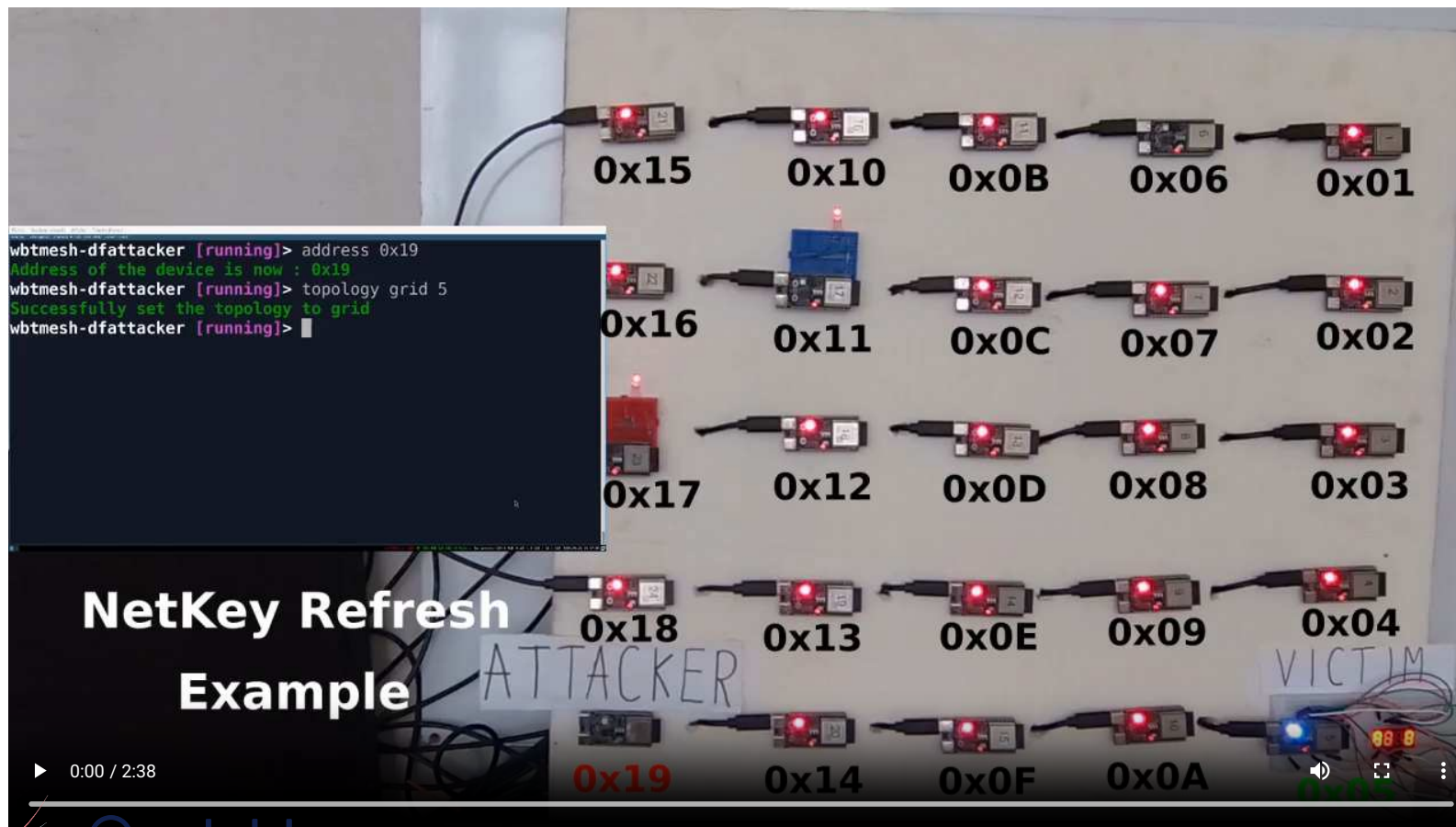


<https://blog.quarkslab.com/modern-tale-blinkenlights.html>

OR CRASHING A SMARTWATCH



POISONING BLUETOOTH MESH DIRECTED FORWARDING

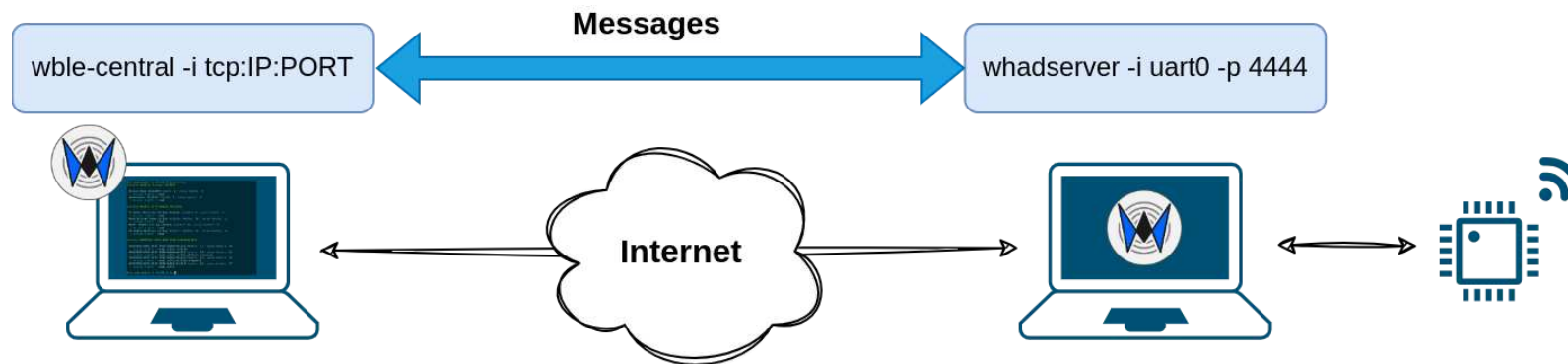


DESYNCHRONIZING WIRELESS HART RECEPTION WINDOW

The slide displays three terminal windows and a photograph of a hardware device.

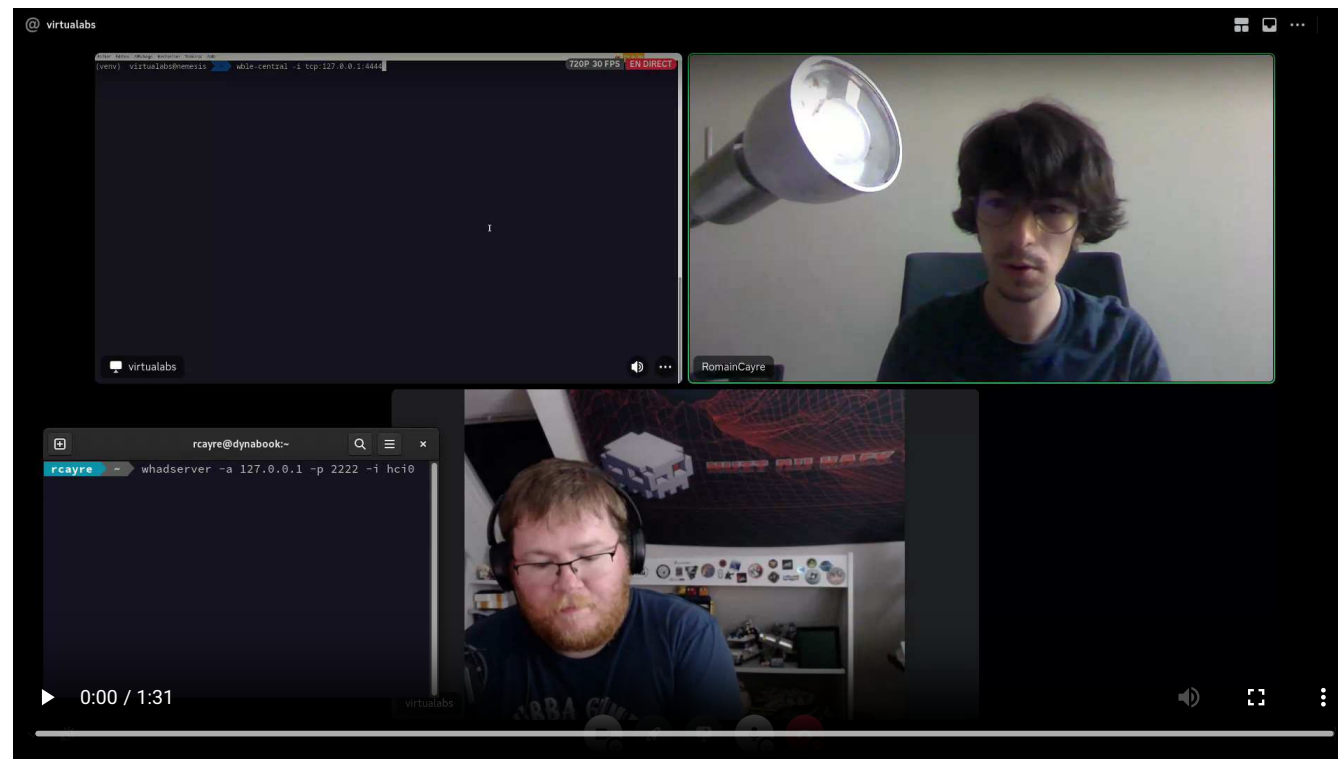
- Mote - Victim**: Shows a series of network packets (MAC, RX, TX) with details like timestamps, sequence numbers, and protocol identifiers. The packets are captured on a specific network interface.
- Whad - Attacker**: Displays a series of network packets, including a decryption failure message and a packet with a timestamp of 419387605. The packets are captured on a specific network interface.
- Manager**: Shows a series of network packets, including a decryption failure message and a packet with a timestamp of 419389499. The packets are captured on a specific network interface.
- Hardware Device**: A photograph of a green circuit board with various components, including a microcontroller, memory chips, and connectors. It is connected to a black cable and a small black box labeled "dust".

PACKETS OVER THE WIRE



- **whadserver** exposes a WHAD device over TCP
- any tool can connect to a WHAD TCP server
- allow relay attacks (with some latency)

USING WHAD HARDWARE FROM 800KM AWAY



CREATING NEW TOOLS

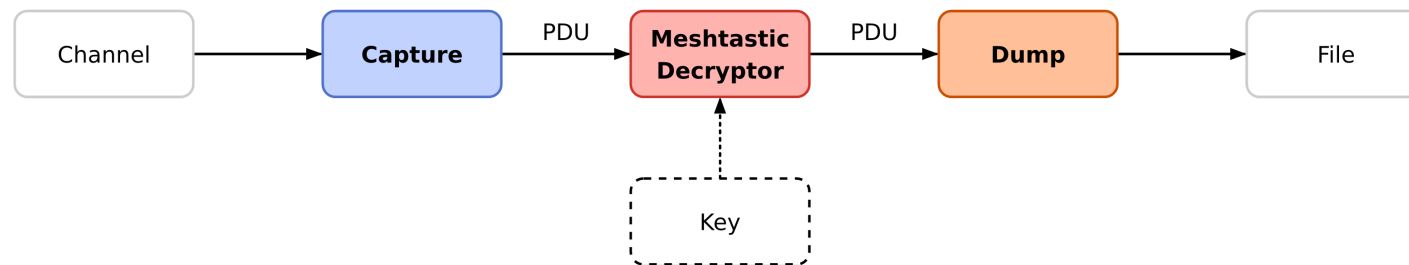
```
from scapy.packet import Packet
from whad.tools.user import user_transform

def ingress(packet: Packet) -> Packet:
    """Process packet received from previous tool"""
    return packet

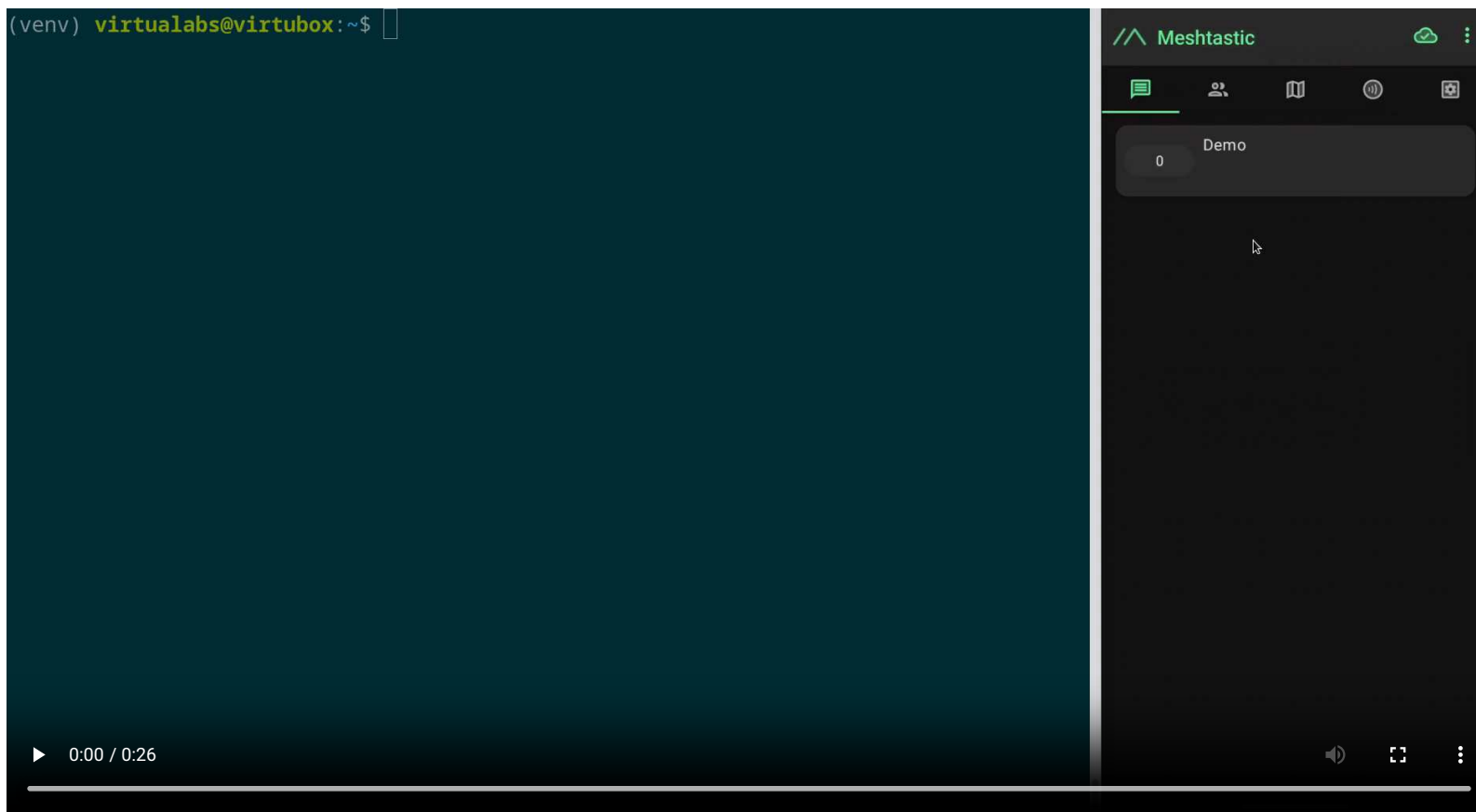
def outgress(packet: Packet) -> Packet:
    """Process a packet received from next tool"""
    return packet

if __name__ == "__main__":
    user_transform(outgress, ingress)
```

DECRYPTING MESHTASTIC PACKETS



```
$ wsniiff -i uart0 phy --lora -f 869525000 -crc -em --preamble-length 8  
↳ -sf 11 -cr 45 -bw 250000 -w 28 |  
↳ python3 meshtastic-decrypt.py -k ENC_KEY |  
↳ wdump meshtastic-decrypt.pcap
```



REAL-TIME HR MONITORING



```
$ wsniff -i uart0 ant | wserver --json --ws
```

```
function initHealthBar() {  
  const socket = new WebSocket('ws://localhost:12345');  
  socket.addEventListener('message', function (event) {  
    /* Extract HR value from packet. */  
    let packet = JSON.parse(event.data);  
    let value = parseInt(packet["ANT_HR_Common_Payload"]["computed_heart_rate"]);  
    /* Update HTML health bar using HR value. */  
    setNumber(value);  
  });  
}
```

SIMULATED WIRELESS LAB

- Designed for hands-on exercises with no specific hardware
- Full emulation of compatible hardware and targets
- User experience is the same, emulated target devices are shown in a browser

SIMULATED WIRELESS LAB



WHAD Workshop

+

⌂

→

↺

↻

http://127.0.0.1:8080/lab/ble_tracker

🔍

☆

👤

📁

☰

GATT permissions

Hack a Bluetooth tracker with WHAD command-line tools (wble-central).

● Server connected



Disconnected

1

Connect to the virtual BLE interface

Use wble-central in interactive mode to connect to the virtual BLE adapter exposed over TCP on port 12345.

```
wble-central -i tcp:127.0.0.1:12345
```

2

Scan and connect to the iTag tracker (ff:ff:30:00:11:22)

Start a BLE scan to discover the iTag device, then connect to it.

Use the 'scan' and 'connect' commands inside the interactive shell.

3

Read the Device Name characteristic

Read the Device Name characteristic (UUID 2A00) from the Generic Access service to retrieve the tracker's advertised name.

Use the 'read' command with characteristic UUID 2A00.

4

Change the device's Device Name characteristic value

Change the Device Name characteristic's value to 'Hacked' by using the 'write' command. Check it had been correctly changed by reading it back.

Scan and connect to the iTag tracker

Start a BLE scan

Inside the `wble-central` interactive shell, start a scan to discover nearby devices:

```
wble-central> scan
```

You should see an advertisement from a device named **iTag**. Note its BD address — you will need it for the next command.

Connect to the device

Once you have spotted the iTag device, connect to it:

```
wble-central> connect ff:ff:30:00:11:22
```

A successful connection looks like:

```
[i] Connected to ff:ff:30:00:11:22
wble-central>
```

Stop Lab

THE PRICE FOR MODULARITY



Complex piping slows down debugging



KISS approach increases fragmentation



Some tools in WHAD are **multi-protocols** (KISS)

CONCLUSION

WHAD IS COOL^{*}

- It helped many researchers building proof-of-concepts and tools
- We use it on a regular basis, track bugs and fix them
- There are a lot of tools available, some of them still ~~undocumented~~ not properly documented

^{*} we are biased

WHAT'S COMING SOON

- **Version 1.3:** support for new wireless protocols and prepping ground for BLE 5.x support
- **Offline lab environment:** provides multiple lab exercises with simulated lab rooms
- **Rust crate** implementing WHAD's foundation in Rust

GIVE WHAD A TRY!

- Need a **flexible library** to write a PoC?
- Want to play with RF packets **the Scapy way**?
- And create tools that **work on any compatible hardware**?

<https://whad.io>

THANK YOU, Q&A TIME !

Romain Cayre

✉ rcayre@laas.fr
📧 @rcayre@infosec.exchange

Damien Cauquil

✉ dcauquil@quarkslab.com
📧 @virtualabs@mamot.fr